Enterprise Agent and Skills Architecture for the Platform

A repository-grounded assessment framework for designing an indispensable, secure and defensible agent ecosystem.

I want to transform this platform into a professional, modern, robust, highly efficient, enterprise-grade product that becomes indispensable to its users—an application they rely on daily and, for critical workflows, throughout the day.

Conduct a thorough analysis of the platform's current architecture, capabilities, target users, business objectives, workflows, constraints, competitive environment, and existing gaps. Based on verified evidence from the repository and available documentation, propose the complete set of specialized AI agents and reusable skills required to bring the platform to this level.

The objective is to achieve the platform's intended vision with the lowest reasonable implementation and maintenance burden—without sacrificing security, reliability, auditability, usability, scalability, or business value.

Strategic objectives

Design an agent-and-skill ecosystem that helps the platform:

- Solve high-frequency, mission-critical user problems.
- Become a daily operating system and trusted source of truth for its customers.
- Automate repetitive work while preserving human control over sensitive decisions.
- Increase user retention, workflow depth, switching costs, and expansion revenue.
- Build defensible product, data, workflow, integration, compliance, and network moats.
- Deliver measurable improvements in productivity, accuracy, visibility, and decision-making.
- Support low-connectivity and operationally constrained environments where applicable.
- Remain secure, explainable, auditable, tenant-safe, and compliant.
- Scale across industries, organization sizes, countries, languages, currencies, and regulatory environments.
- Avoid unnecessary agents, overlapping responsibilities, speculative features, and excessive architectural complexity.

Required analysis

Before proposing solutions:

1. Inspect the repository, documentation, architecture, existing modules, workflows, permissions, integrations, and technical constraints.
2. Identify the platform's expressed product goal and its highest-value user personas.
3. Map the users' daily and hourly jobs, pain points, decisions, exceptions, and bottlenecks.
4. Identify missing capabilities preventing the platform from becoming operationally indispensable.
5. Distinguish problems that require:
    - deterministic application logic;
    - reusable skills;
    - autonomous or semi-autonomous agents;
    - workflow orchestration;

- human approval;
- external integrations.

6. Benchmark the platform conceptually against world-class vertical SaaS and enterprise operating systems.
7. Challenge weak assumptions and identify features that appear attractive but would not create meaningful adoption, retention, or defensibility.

Do not recommend an AI agent where a normal service, rule engine, scheduled job, dashboard, or guided workflow would be safer and simpler.

Agent design requirements

For every proposed agent, define:

- Name and strategic purpose.
- Primary users and jobs to be done.
- Problem solved and expected business value.
- Trigger conditions and operating frequency.
- Inputs, outputs, tools, data sources, and system permissions.
- Skills the agent invokes.
- Memory and context requirements.
- Degree of autonomy.
- Human approval and escalation boundaries.
- Actions the agent may and may not perform.
- Tenant-isolation, privacy, security, and audit requirements.
- Failure modes, abuse cases, and recovery behaviour.
- Dependencies on existing or missing platform capabilities.
- Success metrics and expected effect on adoption, retention, revenue, or operating efficiency.
- MVP scope, mature scope, and features that should explicitly be excluded.

Clearly classify each agent as:

- Advisory
- Monitoring
- Workflow
- Transactional
- Assurance or control
- Orchestration

Skill design requirements

For every proposed reusable skill, define:

- Skill name and responsibility.
- Agent or human users that will invoke it.
- Required inputs and produced outputs.
- Preconditions and validation rules.
- Tools, services, and permissions required.

- Deterministic steps versus AI-assisted reasoning.
- Evidence, citations, and audit records generated.
- Error handling, retry behaviour, and safe failure state.
- Country-, industry-, or tenant-specific configuration.
- Evaluation criteria and acceptance tests.
- Reusability across agents and platform modules.

Skills should be composable, narrowly scoped, testable, observable, versioned, and safe to run repeatedly.

Architecture and governance

Propose an architecture covering:

- Agent orchestration and routing.
- Shared skill registry.
- Tool and permission boundaries.
- Tenant and organization isolation.
- Human-in-the-loop approvals.
- Event-driven and scheduled execution.
- Short-term context and durable memory.
- Prompt and model versioning.
- Structured outputs and validation.
- Idempotency and transaction safety.
- Observability, tracing, cost controls, and performance monitoring.
- Audit trails and evidence retention.
- Evaluation datasets and regression testing.
- Rollback, agent suspension, and incident response.
- Protection against hallucinations, prompt injection, data leakage, unauthorized actions, and excessive autonomy.
- Rules determining when deterministic services must override or constrain model-generated recommendations.

Prioritization

Evaluate and score every proposed agent and skill using:

- User value
- Frequency of use
- Strategic differentiation
- Retention potential
- Revenue potential
- Data-moat potential
- Implementation effort
- Operational cost
- Technical dependency
- Security and compliance risk

- Time to measurable value
- Reusability across the platform

Use a transparent weighted scoring model. Identify assumptions and confidence levels.

Then classify every proposal as:

- Build now
- Build next
- Build later
- Do not build

Required deliverables

Produce:

1. An executive recommendation.
2. A current-state capability and gap assessment.
3. A map of critical daily and hourly user workflows.
4. A complete agent catalogue.
5. A complete reusable-skill catalogue.
6. An agent-to-skill responsibility matrix.
7. A dependency map showing shared services, tools, data, and controls.
8. A prioritized portfolio with weighted scores.
9. A phased implementation roadmap:
    - Foundation
    - First production agents
    - Workflow expansion
    - Intelligence and optimization
    - Ecosystem and defensibility
10. A detailed specification for the first three agents and their supporting skills.
11. A build-versus-buy analysis for models, infrastructure, integrations, and observability.
12. A moat strategy explaining how the system compounds value over time.
13. A risk register with mitigations and release gates.
14. A measurement framework covering:
    - activation;
    - time to first value;
    - workflow completion;
    - agent accuracy;
    - correction and override rates;
    - hours or money saved;
    - daily and weekly active use;
    - retention;
    - expansion revenue;

- operational cost per successful outcome.

15. A 30-, 60-, 90-day execution plan with concrete deliverables, owners, dependencies, and acceptance criteria.

Decision principles

- Base conclusions on repository evidence, not assumptions.
- Surface inconsistencies and missing information.
- Prioritize high-frequency operational workflows over novelty.
- Prefer a small number of excellent agents supported by reusable skills.
- Keep financial, compliance, payroll, inventory, and permission-sensitive truth in deterministic, service-owned systems.
- Require explicit human approval for irreversible, regulated, financial, or high-impact actions.
- Every recommendation must connect to a user problem, measurable outcome, and defensible business advantage.
- Separate genuine competitive moats from features competitors can easily copy.
- Do not implement anything during this assessment unless explicitly instructed.
- Finish with a direct recommendation identifying what should be built first, what should be rejected, and why.